

IAM TASK

1. Create one IAM user and assign EC2 and S3 full access roles.

Objective

To create an IAM user and provide full access permissions to Amazon EC2 and Amazon S3 services.

Steps Performed

Logged in to the AWS Management Console.

Navigated to IAM → Users.

Clicked Create user.

Created a new IAM user with the username: EC2-S3-Admin

Selected Attach policies directly under permissions.

Attached the following AWS managed policies:

AmazonEC2FullAccess

AmazonS3FullAccess

Reviewed the user details and permissions.

Clicked Create user to complete the user creation.

Configuration

IAM User: EC2-S3-Admin

Policies Attached:

AmazonEC2FullAccess

AmazonS3FullAccess

Result

The IAM user EC2-S3-Admin was successfully created with full access to Amazon EC2 and Amazon S3 services.

Evidence

Screenshot 1: IAM user creation

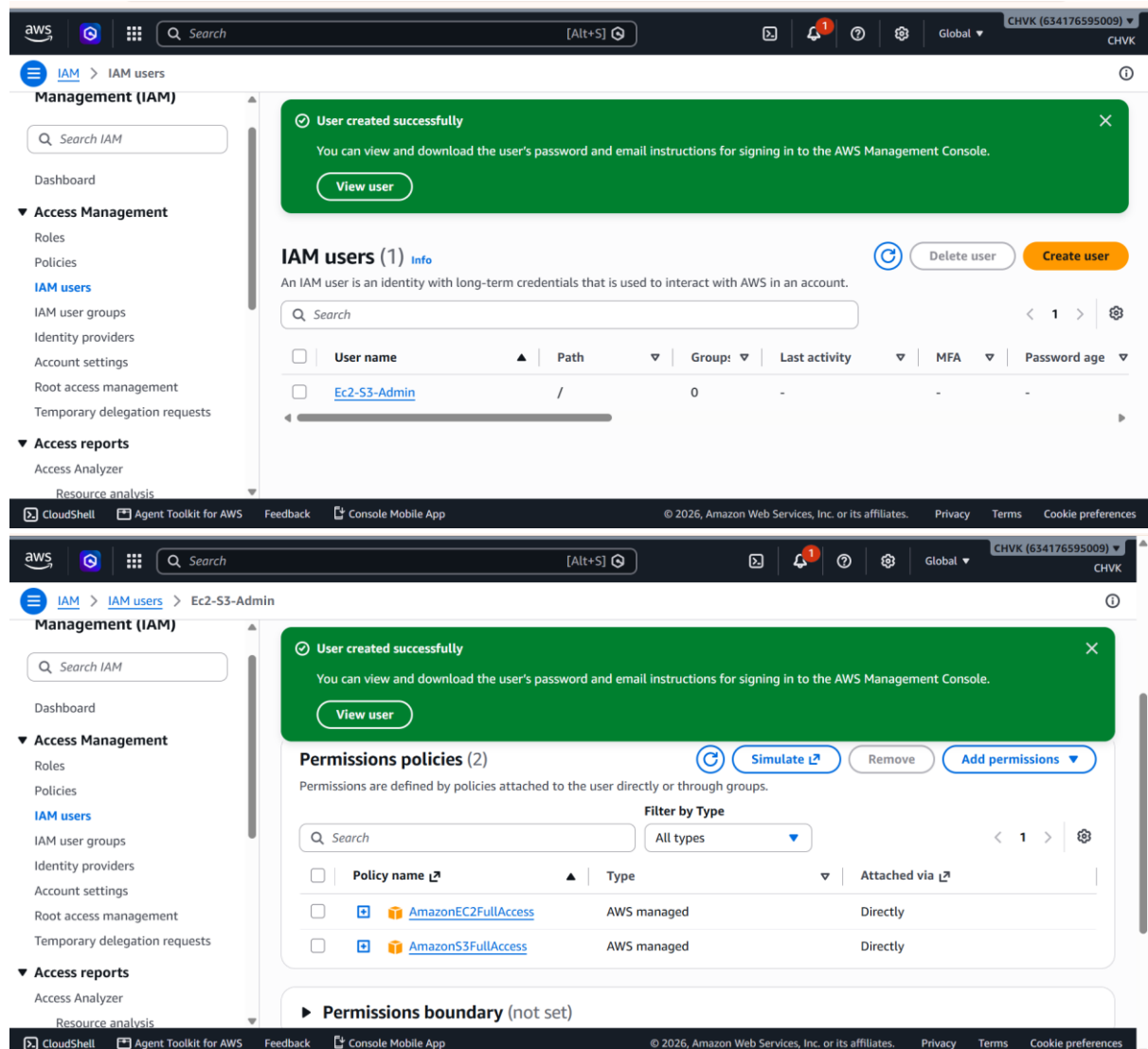
[Insert screenshot here]

Screenshot 2: IAM user permissions

[Insert screenshot showing AmazonEC2FullAccess and AmazonS3FullAccess]

Conclusion

Task 1 was successfully completed. The IAM user was created and the required EC2 and S3 full-access policies were attached.



2. Create one group in IAM and assign read access for EC2.

Objective

To create an IAM group and assign read-only access to Amazon EC2 resources.

Steps Performed

Logged in to the AWS Management Console.

Navigated to IAM → User groups.

Clicked Create group.

Created a new IAM group with the name: EC2-ReadOnly

Under permissions, searched for: AmazonEC2ReadOnlyAccess

Selected and attached the AmazonEC2ReadOnlyAccess policy to the group.

Clicked Create group.

Verified that the policy was successfully attached to the group.

Configuration

IAM Group: EC2-ReadOnly

Policy Attached:

AmazonEC2ReadOnlyAccess

Result

The IAM group EC2-ReadOnly was successfully created with read-only access to Amazon EC2 resources.

Members of this group can view EC2 resources but do not have permissions to create, modify, or delete EC2 resources.

Evidence

Screenshot 1: IAM Group Creation

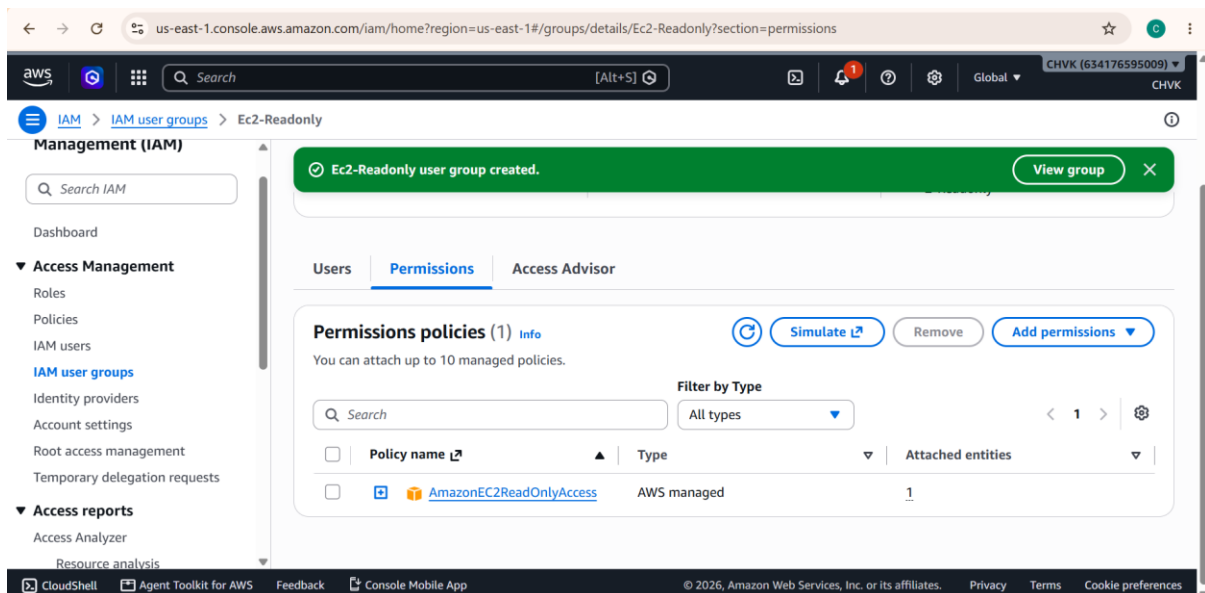
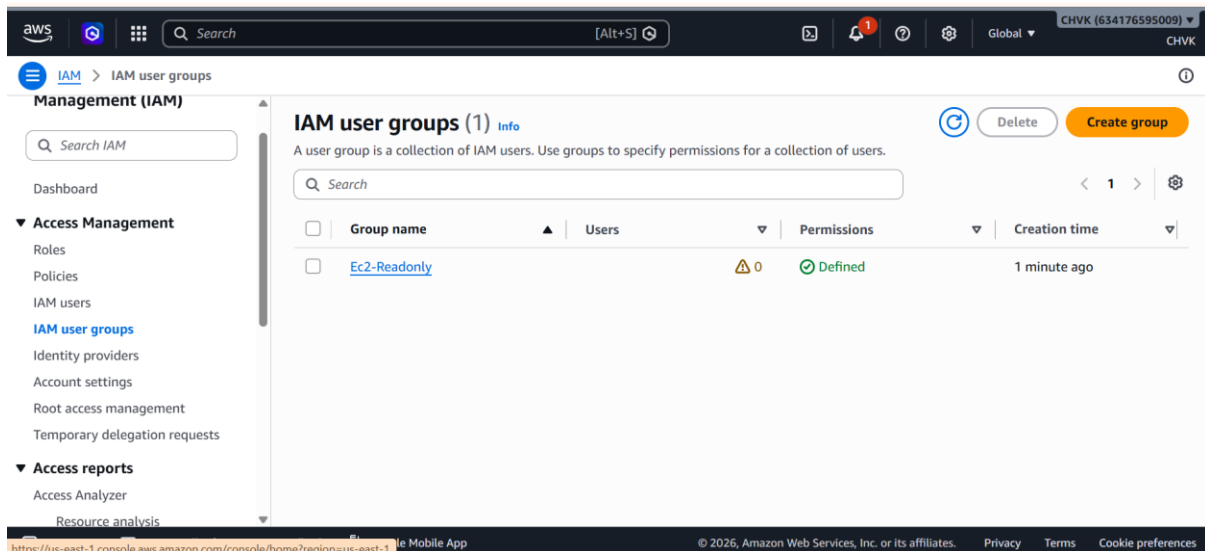
[Insert screenshot here]

Screenshot 2: Group Permissions

[Insert screenshot showing AmazonEC2ReadOnlyAccess]

Conclusion

Task 2 was successfully completed by creating the EC2-ReadOnly IAM group and attaching the AmazonEC2ReadOnlyAccess policy.



3. Create a new user named "Devops" and add to the group created in task 2.

Objective

To create an IAM user named Devops and add the user to the Ec2-ReadOnly group created in Task 2.

Steps Performed

Logged in to the AWS Management Console.

Navigated to IAM → Users.

Clicked Create user.

Created a new IAM user with the username:

Devops

Selected Add user to group under permissions.

Selected the existing IAM group:

Ec2-Readonly

Reviewed the user and group details.

Clicked Create user.

Verified that the Devops user was successfully added to the Ec2-Readonly group.

Verified that the user receives EC2 read-only permissions through the group.

Configuration

IAM User: Devops

IAM Group: Ec2-Readonly

Permission inherited from group: AmazonEC2ReadOnlyAccess

Result

The Devops IAM user was successfully created and added to the Ec2-Readonly group. The user receives EC2 read-only access through the group.

Evidence

Screenshot 1: Devops User Creation

[Insert screenshot here]

Screenshot 2: Devops User Added to Ec2-Readonly Group

[Insert screenshot here]

Screenshot 3: Devops User Permissions

[Insert screenshot showing EC2 read-only permission]

Conclusion

Task 3 was successfully completed by creating the Devops IAM user and adding it to the Ec2-Readonly group. The user inherits EC2 read-only access from the group.

The first screenshot shows the 'IAM users' page in the AWS console. A green notification banner at the top states 'User created successfully' with a 'View user' button. Below the banner, the 'IAM users (2)' section shows a table with two users: 'Devops' and 'Ec2-S3-Admin'. The left sidebar contains navigation links for 'Access Management' and 'Access reports'.

The second screenshot shows the 'Devops' user details page. The 'Groups' tab is selected, showing 'User groups membership' with a table listing 'Ec2-ReadOnly' and 'AmazonEC2ReadOnlyAccess' policies. The 'Permissions' tab is also visible.

The third screenshot shows the 'Permissions policies' tab for the 'Devops' user. It displays a table with one policy: 'AmazonEC2ReadOnlyAccess', which is 'AWS managed' and attached via the 'Ec2-ReadOnly' group. The 'Simulate' button is highlighted.

4. Write a bash script to create an IAM user with VPC full access.

Objective

To create an IAM user using a Bash script and attach the AmazonVPCFullAccess policy to the user.

Steps Performed

Verified that AWS CLI was installed and configured in Git Bash.

Verified AWS identity using: `aws sts get-caller-identity --region us-east-1`

Created a Bash script named: `create-vpc-user.sh`

The script was configured to create an IAM user named: VPC-User

The script attached the AWS managed policy: AmazonVPCFullAccess

Provided execute permission using: `chmod +x create-vpc-user.sh`

Executed the script using: `./create-vpc-user.sh`

The IAM user was successfully created with VPC Full Access.

Verified the attached policy using AWS CLI.

Bash Script

```
#!/bin/bash
```

```
USER_NAME="VPC-User"
```

```
aws iam create-user --user-name "$USER_NAME"
```

```
aws iam attach-user-policy \
```

```
  --user-name "$USER_NAME" \
```

```
  --policy-arn arn:aws:iam::aws:policy/AmazonVPCFullAccess
```

```
echo "IAM user $USER_NAME created with VPC Full Access"
```

Configuration

IAM User: VPC-User

Policy: AmazonVPCFullAccess

Result

The Bash script successfully created the VPC-User IAM user and attached the AmazonVPCFullAccess policy.

Conclusion

Task 4 was successfully completed by creating an IAM user using a Bash script and assigning VPC Full Access permissions.

```

chvka@karthik MINGW64 ~/OneDrive/Desktop
$ aws configure list
NAME      : VALUE                                : TYPE      : LOCATION
profile   : <not set>                                : None      : None
access_key : *****MZON                             : shared-credentials-file :
secret_key : *****Ugg4                             : shared-credentials-file :
region    : east-1                                   : config-file  : ~/.aws/config

chvka@karthik MINGW64 ~/OneDrive/Desktop
$ aws sts get-caller-identity --region us-east-1
{
  "UserId": "634176595009",
  "Account": "634176595009",
  "Arn": "arn:aws:iam:634176595009:root"
}

chvka@karthik MINGW64 ~/OneDrive/Desktop
$ nano create-vpc-user.sh

chvka@karthik MINGW64 ~/OneDrive/Desktop
$ chmod +x create-vpc-user.sh

chvka@karthik MINGW64 ~/OneDrive/Desktop
$ ./create-vpc-user.sh
{
  "User": {
    "Path": "/",
    "UserName": "VPC-User",
    "UserId": "AIDAZHJ664RAYJIWCPGSK",
    "Arn": "arn:aws:iam:634176595009:user/VPC-User",
    "CreateDate": "2026-09-02T09:30:06+00:00"
  }
}
IAM user VPC-User created with VPC Full Access

```

5. Create an IAM policy to allow EC2 access for a specific user in specific regions only.

Objective

To create a custom IAM policy that allows EC2 access only in the specified AWS regions.

Allowed Regions

us-east-1

us-west-2

Steps

Open AWS Console → IAM → Policies.

Click Create policy.

Select the JSON editor.

Created a custom policy named: EC2-Specific-Regions-Access

Added the following policy:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "ec2:*",
      "Resource": "*",
      "Condition": {

```

```
"StringEquals": {
  "aws:RequestedRegion": [
    "us-east-1",
    "us-west-2"
  ]
},
{
  "Effect": "Deny",
  "Action": "ec2:*",
  "Resource": "*",
  "Condition": {
    "StringNotEquals": {
      "aws:RequestedRegion": [
        "us-east-1",
        "us-west-2"
      ]
    }
  }
}
```

Attached the policy to the IAM user VPC-User.

Verified that VPC-User has:

AmazonVPCFullAccess

EC2-Specific-Regions-Access

Verification

The following command was tested for the allowed region:

```
aws ec2 describe-instances --region us-east-1
```

The command successfully returned EC2 instance information, confirming EC2 access in us-east-1.

The following command was tested for ap-south-1:

```
aws ec2 describe-instances --region ap-south-1
```

Initially, it returned:

```
{  
  "Reservations": []  
}
```

This did not confirm access denial because the AWS CLI was authenticated using the root account.

Final Verification

The policy should be verified using VPC-User credentials:

```
aws sts get-caller-identity --region us-east-1
```

The ARN should show:

```
arn:aws:iam::<ACCOUNT-ID>:user/VPC-User
```

Then test:

```
aws ec2 describe-instances --region us-east-1
```

Expected: Access allowed 

```
aws ec2 describe-instances --region ap-south-1
```

Expected: UnauthorizedOperation / AccessDenied 

Result

A custom IAM policy was created to restrict EC2 access to the specified AWS regions. Final restriction verification must be performed using the VPC-User credentials rather than root credentials.

Evidence / Screenshots

EC2-Specific-Regions-Access policy JSON.

Policy attached to VPC-User.

us-east-1 successful EC2 command.

ap-south-1 AccessDenied/UnauthorizedOperation using VPC-User.

aws [Search] [Alt+S] CHVK (634176595009) CHVK

Identity and Access Management (IAM)

Policy Ec2-Specific-Regions-Access created. [View policy](#)

Policies (1571)

A policy is an object in AWS that defines permissions.

Filter by Type: Customer managed 1 match

Policy name	Type	Used as
Ec2-Specific-Regions-Access	Customer managed	None

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

aws [Search] [Alt+S] CHVK (634176595009) CHVK

Identity and Access Management (IAM)

1 policy added

Permissions Groups Tags Security credentials Last Accessed

Permissions policies (2)

Permissions are defined by policies attached to the user directly or through groups.

Filter by Type: All types

Policy name	Type	Attached via
AmazonVPCFullAccess	AWS managed	Directly
Ec2-Specific-Regions-Access	Customer managed	Directly

Permissions boundary (not set)

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

```
chvka@karthik MINGW64 ~/OneDrive/Desktop
$ aws configure
AWS Access Key ID [*****]: AKIAZHJ664RATH4PCLMP
AWS Secret Access Key [*****]: 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
Default region name [us-east-1]: us-east-1
Default output format [json]: json

chvka@karthik MINGW64 ~/OneDrive/Desktop
$ aws sts get-caller-identity --region us-east-1
{
  "UserId": "AIDAZHJ664RAYJIWCPGSK",
  "Account": "634176595009",
  "Arn": "arn:aws:iam:634176595009:user/VPC-User"
}

chvka@karthik MINGW64 ~/OneDrive/Desktop
$ aws ec2 describe-instances --region us-east-1
{
  "Reservations": [
    {
      "ReservationId": "r-0e8470c7a344a7ca6",
      "OwnerId": "634176595009",
      "Groups": [],
      "Instances": [
        {
          "Architecture": "x86_64",
          "BlockDeviceMappings": [
            {
              "DeviceName": "/dev/sda1",
              "Ebs": {
                "AttachTime": "2026-08-10T08:49:18+00:00",
                "DeleteOnTermination": true,
                "Status": "attached",
                "VolumeId": "vol-0da942ce617826cdc",
                "EbsCardIndex": 0
              }
            }
          ],
          "ClientToken": "f8bdbb78-bd7b-447c-a183-1277067a5682",
          "EbsOptimized": true,
          "EnaSupport": true,
          "Hypervisor": "xen",
          "NetworkInterfaces": [
            {
              "Attachment": {
                "AttachTime": "2026-08-10T08:49:17+00:00",
                "AttachmentId": "eni-attach-0d6437fd6f63dafa86",

```

```
chvka@karthik MINGW64 ~/OneDrive/Desktop
$ aws ec2 describe-instances --region ap-south-1
aws: [ERROR]: An error occurred (UnauthorizedOperation) when calling the DescribeInstances operation: You are not authorized to perform this operation. User: arn:aws:iam:634176595009:user/VPC-User is not authorized to perform: ec2:DescribeInstances with an explicit deny in an identity-based policy: arn:aws:iam:634176595009:policy/Ec2-Specific-Regions-Access
```

6. We have two accounts: Account A and Account B. Account A user should access an S3 bucket in Account B.

(Collaborate with a team member and execute this. This is mostly asked in every interview.)

Steps I Used

1. Identified the two AWS accounts

- Account A:331116047691 — my account.
- Account B:634176595009 — Karthik's account.

2. Used the existing IAM user Asif in Account A

- I used the existing Asif IAM user instead of creating a new user..
- Created an access key for Asif so I could test the access using AWS CLI..

3. Created an S3 bucket in Account B

- Bucket name:

asif-karthik-cross-account-2026

- The bucket was created in Karthik's AWS account.

4. Added a bucket policy in Account B

The bucket policy allowed the Asif user from Account A:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowAsifAccess",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::331116047691:user/Asif"
      },
      "Action": [
        "s3:ListBucket",
        "s3:GetObject",
        "s3:PutObject"
      ]
    }
  ]
}
```

```

    ],
    "Resource": [
        "arn:aws:s3:::asif-karthik-cross-account-2026",
        "arn:aws:s3:::asif-karthik-cross-account-2026/*"
    ]
}
]
}

```

This allowed the Asif user from Account A to list the bucket and read/write objects.

5. Added an IAM policy to Asif in Account A

I created the inline policy CrossAccount-S3-Access:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowListBucket",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::asif-karthik-cross-account-2026"
    },
    {
      "Sid": "AllowObjects",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::asif-karthik-cross-account-2026/*"
    }
  ]
}

```

I
}

6. Configured a separate AWS CLI profile

I created the profile:

```
aws configure --profile asif-cross-account
```

I configured the credentials for the Asif user and selected ap-south-1 as the region.

7. Verified the AWS identity

```
aws sts get-caller-identity --profile asif-cross-account
```

Output confirmed:

Account: 331116047691

Arn: arn:aws:iam::331116047691:user/Asif

This confirmed that the CLI was using the correct Asif user.

8. Tested access to the bucket

```
aws s3 ls s3://asif-karthik-cross-account-2026 \
```

```
--profile asif-cross-account \
```

```
--region ap-south-1
```

The command completed without AccessDenied, confirming that Asif could access the bucket.

9. Created a test file

```
echo "Cross account S3 test" > test.txt
```

10. Uploaded the file to Account B's bucket

```
aws s3 cp test.txt s3://asif-karthik-cross-account-2026/ \
```

```
--profile asif-cross-account \
```

```
--region ap-south-1
```

Output:

```
upload: .\test.txt to s3://asif-karthik-cross-account-2026/test.txt
```

This successfully proved cross-account S3 access.

Conclusion

Cross-account S3 access was successfully configured between two AWS accounts. IAM user Asif from Account A (331116047691) was granted permission to access the S3 bucket asif-

karthik-cross-account-2026 located in Account B (634176595009). The configuration was successfully validated by identifying the correct IAM user, listing the bucket, and uploading test.txt from Account A to the S3 bucket in Account B. Therefore, Task 6 was completed successfully.

The first screenshot shows the AWS console for the bucket 'asif-karthik-cross-account-2026'. A green notification bar at the top states 'Successfully created bucket "asif-karthik-cross-account-2026"'. The bucket's 'Objects' tab is selected, showing 'Objects (0)'. A table below the header indicates 'No objects' and 'You don't have any objects in this bucket.' with an 'Upload' button.

The second screenshot shows the 'Bucket policy' editor. A green notification bar states 'Successfully edited bucket policy.' Below it, a blue information box notes: 'Public access is blocked because Block Public Access settings are turned on for this bucket'. The JSON policy is displayed in a text area, and a 'Copy' button is visible.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowAsifAccess",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::331116047691:user/Asif"
      },
      "Action": [
        "s3:ListBucket",
        "s3:GetObject",
        "s3:PutObject"
      ]
    }
  ]
}
```

The third screenshot shows the 'Specify permissions' step in the IAM console. The 'Policy editor' is open, showing a JSON policy with two statements. The first statement allows 's3:ListBucket' on the bucket 'asif-karthik-cross-account-2026'. The second statement allows 's3:GetObject' and 's3:PutObject' on the same bucket.

```
1 *
2 "Version": "2012-10-17",
3 "Statement": [
4 {
5   "Sid": "AllowCrossAccountBucketAccess",
6   "Effect": "Allow",
7   "Action": [
8     "s3:ListBucket"
9   ],
10  "Resource": "arn:aws:s3::asif-karthik-cross-account-2026"
11 },
12 {
13   "Sid": "AllowCrossAccountObjectAccess",
14   "Effect": "Allow",
15   "Action": [
16     "s3:GetObject",
17     "s3:PutObject"
18   ],
19   "Resource": "arn:aws:s3::asif-karthik-cross-account-2026/*"
20 }
21 ]
22 }
```

aws [Alt+S] Ask Amazon Q Shaik Asif (33116047691) Shaik%20Asif

management (IAM)

Search IAM

Dashboard

Access Management

Roles

Policies

IAM users

IAM user groups

Identity providers

Account settings

Root access management

Temporary delegation requests

Access reports

Access Analyzer

Resource analysis

Unused access

Analyzer settings

Policy simulator

Credential report

Multi-account access

CloudShell Agent Toolkit for AWS Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Asif Info

Summary

ARN
arn:aws:iam::33116047691:user/Asif

Console access
Disabled

Access key 1
[Create access key](#)

Created
September 02, 2026, 15:49 (UTC+05:30)

Last console sign-in
-

Permissions Groups Tags Security credentials Last Accessed

Permissions policies (1)

Permissions are defined by policies attached to the user directly or through groups.

Search Filter by Type All types

Policy name	Type	Attached via
CrossAccount-S3-Access	Customer inline	Inline

Permissions boundary (not set)

```
asifs@Asif_Shaik MINGW64 ~
$ aws configure --profile asif-cross-account
AWS Access Key ID [*****]: AKIAU2GAN5VFVJ62WI6W
AWS Secret Access Key [*****]: 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
Default region name [ap-south-1]: ap-south-1
Default output format [json]: json

asifs@Asif_Shaik MINGW64 ~
$ aws sts get-caller-identity --profile asif-cross-account
{
  "UserId": "AIDAU2GAN5VFTUXQRRWUW",
  "Account": "33116047691",
  "Arn": "arn:aws:iam::33116047691:user/Asif"
}

asifs@Asif_Shaik MINGW64 ~
$ aws s3 ls s3://asif-karthik-cross-account-2026 --profile asif-cross-account --region ap-south-1

asifs@Asif_Shaik MINGW64 ~
$ aws s3 cp test.txt s3://asif-karthik-cross-account-2026/ --profile asif-cross-account --region ap-south-1
upload: .\test.txt to s3://asif-karthik-cross-account-2026/test.txt
```